Informe TP: Motor de búsqueda de funciones similares vía AST

1. Diseño

Comparamos funciones de Python por su estructura y no por su texto: el AST que genera `ast.parse` es el árbol de derivación de la gramática de Python, así que comparar ASTs es comparar árboles de derivación, no strings. El sistema tiene cuatro módulos:

1. **Extracción** (`extraccion.py`): `os.walk` más `ast.parse` sobre cada `.py`; devuelve toda `FunctionDef` y `AsyncFunctionDef` con nombre, archivo, línea y subárbol (saltea archivos con errores de sintaxis).
2. **Normalización** (`normalizacion.py`): un `ast.NodeTransformer` que elimina los nombres propios de cada función.
3. **Serialización** (`similitud.py`): recorrido pre-order recursivo con `ast.iter_child_nodes` (`ast.walk` va por niveles) que produce una lista de tokens: tipo de nodo más el valor normalizado en `Name`, `arg` y `Constant`.
4. **Similitud y búsqueda** (`similitud.py`, `buscar_similares.py`): compara la función objetivo contra todas y devuelve las top-K por score. CLI: `--dir`, `--funcion archivo.py:nombre`, `--top` y `--metrica`.

2. Normalización

La normalización hace que funciones con la misma estructura queden idénticas aunque difieran en nombres y valores. Trabaja sobre una copia (`copy.deepcopy`), porque `NodeTransformer` modifica el árbol in situ.

- **Variables y parámetros** (`Name`, `arg`): `VAR1`, `VAR2`, ... por orden de primera aparición, con un diccionario por función.
- **Literales**: números y strings pasan a `LIT`.
- **Nombre de la función**: `FUNC`. También se descartan las anotaciones de tipo y los **docstrings**.
- **Builtins** (`print`, `len`, `range`...): se conservan sin normalizar (`dir(builtins)`), para que `print(x)` y `len(x)` no se vean iguales.

3. Métrica y umbral

La métrica principal es la similitud de Jaccard sobre trigramas (n=3) de la secuencia de tokens, con umbral 0.40 y costo O(L) por par de funciones. Como alternativa implementamos la distancia de Levenshtein normalizada (`1 - distancia / max(len(a), len(b))`, costo $O(L^2)$), con umbral 0.45, seleccionable con `--metrica`.

Por qué n-gramas y por qué n=3. Con tokens sueltos, dos funciones con los mismos nodos en distinto orden darían 1.0; los n-gramas capturan qué nodo sigue a cuál y toleran el reordenamiento de líneas. En un primer conjunto de 6 funciones probamos n=2, 3 y 4: n=2 subió los no clones a 0.29 por coincidencias casuales (por ejemplo `Return → Name`), n=4 bajó el clon reordenado a 0.32, cerca del ruido, y n=3 dejó los no clones en 0.11 o menos y los clones en 0.48 o más.

Validación con 17 funciones (objetivo: `suma_lista`; umbrales: Jaccard 0.40, Levenshtein 0.45). Ampliamos `ejemplos/` con casos más difíciles.

Grupo	Funciones	Jaccard (n=3)	Levenshtein
Clones exactos tras normalizar	`acumular`, `suma_con_base`, `suma_documentada`	1.00	1.00
Clones con cambios	`suma_positivos` / `suma_y_cuenta` / `suma_con_while`	0.70 / 0.48 / 0.33	0.80 / 0.56 / 0.52
Clon semántico	`suma_builtin`	0.09	0.24
No clones, mismo esqueleto	`invertir_cadena` / `contar_pares` / `factorial` / `maximo`	0.75 / 0.49 / 0.47 / 0.37	0.94 / 0.66 / 0.74 / 0.67
No clones distintos	`fibonacci`, `buscar_indice`, `saludar`, `es_par`, `promedio`	0.26 a 0.06	0.57 a 0.24

Precisión y recall. Se consideran clones las 6 funciones estructuralmente equivalentes (los dos primeros grupos de la tabla); `suma_builtin`, clon semántico, queda fuera del recall.

Métrica y umbral	Marcadas como candidatas	Precisión	Recall
Jaccard 0.30	10	0.60	1.00
Jaccard 0.40 (elegido)	8	0.63	0.83
Jaccard 0.50	5	0.80	0.67
Levenshtein 0.45	11	0.55	1.00

Ya no existe un umbral que separe clones de no clones. Jaccard con 0.40 es un compromiso: detecta 5 de 6 clones con 3 falsos positivos (`invertir_cadena`, `contar_pares`, `factorial`); subir el umbral pierde clones y bajarlo suma falsos positivos. Levenshtein separa peor (`invertir_cadena` 0.94 supera a `suma_y_cuenta` 0.56), por eso Jaccard es la métrica por defecto. La causa es la normalización: muchas funciones comparten el esqueleto «inicializar, `for`, actualizar, `return`», así que la herramienta sirve como filtro de candidatas para revisión humana, priorizando no perder clones. Los umbrales son estimaciones (un único objetivo, 17 funciones).

4. Limitaciones conocidas

- **Falsos positivos por esqueleto compartido**: `invertir_cadena` 0.75, `contar_pares` 0.49 y `factorial` 0.47 contra `suma_lista` (Jaccard).
- **Clones que no detecta**: con otra estructura de bucle (`suma_con_while`, 0.33) o semánticos (`suma_builtin`, 0.09).
- **Normalización incompleta**: las funciones propias llamadas se normalizan como variables, los atributos (`self.total`) no se normalizan.
- **Alcance e implementación**: reordenar líneas baja el score (`suma_y_cuenta` da 0.48); solo se comparan funciones completas; la CLI no permite pegar el código y la búsqueda es $O(n^2)$.

5. Mejoras posibles

- **Ponderar los n-gramas** (TF-IDF) para bajar falsos positivos como `invertir_cadena`.
- **Canonicalizar construcciones equivalentes** (`for`/`while`), normalizar atributos y comparar sub-bloques.
- **Validar los umbrales** con repositorios reales, agregar `--codigo` (código objetivo desde un archivo) y escalar la búsqueda con MinHash.